AWS 실무 개발자 레퍼런스

기능별로 무엇인지, 어떤 상황에서 쓰는지, 그리고 처음 쓸 때 대부분 밟는 지뢰가 무엇인지. 자격증 범위가 아니라 실제 서비스를 굴릴 때 마주치는 것만 골랐다.

32개 항목 · 7개 카테고리 · 수치 검증일 2026-07-29

목차

01 컴퓨트 — 코드를 어디서 돌릴 것인가

02 네트워크 — 트래픽이 들어오는 길

03 스토리지 — 파일을 어디에 둘 것인가

04 데이터베이스 — 상태를 어디에 저장할 것인가

05 메시징·이벤트 — 서비스를 어떻게 분리할 것인가

06 보안·자격증명 — 누가 무엇을 할 수 있는가

07 배포·관측 — 어떻게 올리고 어떻게 볼 것인가

08 비용 함정 정리

01 / COMPUTE

컴퓨트 — 코드를 어디서 돌릴 것인가

선택 기준은 성능이 아니라 *운영 책임을 어디까지 넘길 것인가*다. OS 패치까지 직접 하면 EC2, 컨테이너 이미지까지만 책임지면 Fargate, 함수 코드만 책임지면 Lambda.

컴퓨트 선택 요약

선택	내가 관리하는 것	맞는 상황	안 맞는 상황
Lambda	함수 코드만	이벤트 기반, 간헐적 트래픽, 15분 내 완료	상시 고부하, 긴 배치, 대용량 인메모리 캐시
ECS + Fargate	컨테이너 이미지	보통의 웹 API·워커. AWS 안에서만 돌 때	K8s 생태계 도구가 이미 필수일 때
EKS	컨테이너 + K8s 설정	멀티클라우드/온프렘 이식성, Helm·Operator 자산	팀에 K8s 운영 인력이 없을 때
EC2	OS · 런타임 · 앱 전부	특수 커널·GPU·라이선스, 리프트앤시프트	단순 웹앱. 운영 부담만 늘어난다

EC2 Elastic Compute Cloud 리전 · AZ

가상 서버. 인스턴스 타입 이름은 규칙이 있다 — `c7gn.xlarge`는 **c**(compute optimized) **7**세대 **g**(Graviton/ARM) **n**(네트워크 강화), 크기 xlarge.

계열 — `t` 버스터블(개발·저부하) · `m` 범용 · `c` CPU 집약 · `r` / `x` 메모리 집약 · `i` 로컬 NVMe · `g` / `p` GPU

언제 — 커널 모듈이 필요하거나, 상용 DB 라이선스가 인스턴스에 묶여 있거나, 기존 온프렘 서버를 그대로 옮길 때.

함정 — **t 계열의 CPU 크레딧.** 베이스라인을 넘겨 쓰면 크레딧이 소진되고 성능이 절벽처럼 떨어진다. `unlimited` 모드면 안 떨어지는 대신 추가 과금된다. 프로덕션 API를 t3.micro로 돌리다 원인 모를 지연을 겪는 사례가 가장 흔하다.

Lambda 서버리스 함수 실행 리전

이벤트가 오면 컨테이너를 띄워 함수를 실행하고 내린다. 요청 수와 실행 시간(GB-초)으로 과금되므로 *트래픽이 없으면 0원*이다.

한도 타임아웃 최대 15분(기본 3초) · 메모리 128MB–10,240MB · `/tmp` 512MB–10GB · 동기 페이로드 6MB, 비동기 1MB · 리전당 기본 동시 실행 1,000

언제 S3 업로드 후처리, SQS 소비, 크론 배치, API Gateway 백엔드, 이벤트 라우팅의 접착제.

합정 **콜드 스타트와 VPC.** Lambda를 VPC에 붙이면 ENI가 필요해 초기화가 느려진다(현재는 Hyperplane ENI로 많이 개선됐지만 여전히 0은 아니다). 그리고 **동시 실행 1,000은 계정·리전 공유 자원**이다. 한 함수가 폭주하면 같은 계정의 다른 함수까지 스로틀된다 — 중요 함수엔 예약 동시성(reserved concurrency)을 걸어라.

ECS + Fargate Elastic Container Service 리전

AWS 자체 컨테이너 오케스트레이터. Fargate는 그 아래 깔리는 서버리스 실행 엔진이라 EC2 노드를 직접 관리하지 않아도 된다. Fargate는 ECS와 EKS 양쪽 모두에서 쓸 수 있다.

구조 Task Definition(컨테이너 스펙) → Task(실행 단위) → Service(원하는 개수 유지 + ALB 연결)

언제 K8s를 쓸 이유가 딱히 없는 팀의 기본값. IAM 롤을 태스크 단위로 붙일 수 있어 권한 분리가 쉽다.

합정 **Fargate는 vCPU·메모리 조합이 정해져 있다.** 0.25 vCPU에 8GB 같은 구성은 불가능하다. 메모리만 크게 필요한 워크로드는 CPU를 낭비하며 비싸진다. 상시 풀가동 워크로드는 EC2 시작 유형 + Savings Plan이 대체로 더 싸다.

EKS Elastic Kubernetes Service 리전

관리형 Kubernetes 컨트롤 플레인. AWS가 마스터 노드를 운영하고, 워커 노드는 EC2 노드그룹이나 Fargate 프로파일로 붙인다.

언제 이미 Helm 차트·Operator·GitOps 파이프라인이 자산으로 있을 때, 또는 온프렘/타 클라우드와 동일한 매니페스트를 써야 할 때.

합정 **"관리형"은 컨트롤 플레인만이다.** 노드 AMI 업그레이드, CNI 플러그인 버전, IRSA 설정, 클러스터 버전 업그레이드는 전부 내 몫이다. K8s는 대략 1년에 3번 마이너 버전을 낸다. 이걸 감당할 사람이 없으면 ECS가 정답이다.

02 / NETWORKING

네트워크 — 트래픽이 들어오는 길

AWS 장애 원인 1위는 코드가 아니라 네트워크 설정이다. VPC·보안그룹·라우팅 테이블 세 개만 정확히 이해해도 디버깅 시간의 절반이 줄어든다.

VPC Virtual Private Cloud 리전

내 계정 전용 가상 네트워크. 실무에서 실제로 만지는 건 다섯 조각이다.

구성

- **서브넷** — AZ 하나에 묶인 IP 대역. 라우팅 테이블이 인터넷 게이트웨이를 가리키면 퍼블릭, 아니면 프라이빗.
- **IGW** — VPC의 인터넷 출입구. 없으면 퍼블릭 IP가 있어도 통신 안 된다.
- **NAT Gateway** — 프라이빗 서브넷이 *바깥으로만* 나갈 때 쓴다. 안에서 밖은 되고 밖에서 안은 안 된다.
- **라우팅 테이블** — 서브넷별로 "이 대역은 어디로 보낼지".
- **VPC 엔드포인트** — S3·DynamoDB 등 AWS 서비스를 인터넷 안 거치고 부르는 통로.

설계

최소 2개 AZ에 퍼블릭/프라이빗 서브넷 쌍. ALB는 퍼블릭, 앱과 DB는 프라이빗. 이게 사실상 표준 3티어 구성이다.

함정

> **NAT Gateway가 조용히 제일 비싼 줄이 된다.** 시간당 `$0.045` + 처리 데이터 GB당 `$0.045`(us-east-1). AZ마다 하나씩 두면 고정비만 월 `$100`을 넘고, 여기에 S3로 오가는 트래픽까지 전부 GB 과금된다. **S3·DynamoDB는 Gateway 엔드포인트가 무료**이니 반드시 먼저 붙여라. (Interface/PrivateLink 엔드포인트는 유료다.)

Security Group / NACL 방화벽 두 계층 VPC

둘 다 방화벽이지만 동작이 다르다. 실무에서는 보안 그룹만 쓰고 NACL은 기본값으로 두는 경우가 대부분이다.

	Security Group	Network ACL
적용 대상	ENI(인스턴스) 단위	서브넷 단위
상태	Stateful — 나간 응답은 자동 허용	Stateless — 인·아웃 규칙을 각각 써야 함
규칙	허용만 가능 (Deny 없음)	허용 + 거부, 번호 순서대로 평가
쓰는 때	기본. 90%는 여기서 끝난다	특정 IP 대역 차단 등 명시적 Deny가 필요할 때

합정

> **보안 그룹은 소스로 다른 보안 그룹을 지정할 수 있다.** DB 보안 그룹의 인바운드를 `0.0.0.0/0`이나 VPC CIDR로 열지 말고 `sg-app`으로 지정하라. IP가 바뀌어도 깨지지 않고, 침해 시 확산 범위도 줄어든다. NACL을 손댔다면 **돌아오는 트래픽의 임시 포트(1024–65535) 아웃바운드**를 안 열어서 생기는 미스터리한 타임아웃을 항상 의심하라.

ELB Elastic Load Balancing — ALB / NLB / GWLB 리전

세 종류가 각각 OSI 다른 계층에서 동작한다. 이름이 아니라 계층으로 고르면 헷갈리지 않는다.

종류	계층	할 수 있는 것	고르는 이유
ALB	L7 (애플리케이션)	경로/호스트 기반 라우팅, HTTP 헤더 조작, Cognito 인증 연동, WAF 연결	HTTP(S) API·웹의 기본값
NLB	L4 (전송)	TCP/UDP/TLS, 초고성능, 고정 IP(EIP) 부여 가능	gRPC·게임·MQTT, 또는 화이트리스트용 고정 IP가 필요할 때
GWLB	L3 (네트워크)	GENEVE(6081)로 서드파티 방화벽·IDS 어플라이언스에 트래픽 투과	보안 어플라이언스를 인라인으로 끼울 때

합정

> **ALB의 유휴 타임아웃 기본값은 60초**다. 백엔드 처리가 60초를 넘으면 ALB가 먼저 끊고 클라이언트는 `504`를 본다. 긴 작업은 타임아웃을 늘리는 게 아니라 비동기(SQS + 폴링)로 바꾸는 게 정석이다. 또 ALB/NLB는 **트래픽이 0이어도 시간당 고정비**가 붙는다 — 개발용으로 띄워두고 잊지 마라.

Route 53 관리형 DNS 글로벌

DNS 서비스이자 사실상 트래픽 분배 계층. 이름의 53은 DNS 포트 번호에서 왔다. 호스팅 영역은 `$0.50`/월(25개까지)이다.

라우팅 정책

Simple · Weighted(카나리·A/B) · Latency(가장 빠른 리전) · Failover(헬스체크 기반 DR) · Geolocation(국가별) · Geoproximity · Multivalue answer · IP-based — 8종.

언제

Weighted로 신버전에 5%만 흘려보내는 카나리 배포, Failover로 리전 장애 시 DR 리전 전환, ALIAS 레코드로 ALB·CloudFront·S3에 무료 연결.

합정

> **TTL을 잊고 배포하면 롤백이 안 먹는다.** TTL 300초짜리 레코드를 바꿔놓고 "왜 아직 옛날 서버로 가지?"를 30분 동안 디버깅하는 일이 흔하다. 전환 예정이 있으면 *미리* TTL을 60초로 낮춰두고 작업하라. 그리고 CNAME은 zone apex(`example.com`)에 못 쓴다 — AWS 리소스라면 **ALIAS**를 써야 한다.

`CloudFront` CDN 글로벌 엣지

전 세계 엣지 로케이션에 콘텐츠를 캐싱해 지연을 줄인다. 정적 파일 가속만이 아니라 *오리진 앞의 보호막*으로도 쓴다.

언제 — S3 정적 사이트 배포(+OAC로 버킷 비공개 유지), API 앞단 캐싱, WAF·Shield 적용 지점, 오리진 그룹으로 장애 시 보조 오리진 페일오버.

합정 — **오리진 페일오버는 GET·HEAD·OPTIONS에만 동작한다.** POST/PUT은 넘어가지 않으므로 쓰기 API의 DR 용도로 착각하면 안 된다. 그리고 **캐시 무효화(invalidation)는 월 1,000경로까지만 무료**다. 배포마다 `/*`를 날리는 습관 대신 파일명에 해시를 넣는 버전 관리가 정공법이다.

`API Gateway` 관리형 API 프론트도어 리전

인증·스로틀링·요청 검증·변환을 앞단에서 처리하고 Lambda나 백엔드로 넘긴다. REST API와 HTTP API 두 종류가 있고 성격이 꽤 다르다.

	HTTP API	REST API
가격	더 저렴	더 비쌈
통합 타임아웃	최대 30초 (상향 불가)	기본 최대 29초. Regional·Private는 Service Quotas로 **상향 가능**
기능	JWT 오서라이저, 기본 기능 위주	요청/응답 매핑 템플릿, WAF, 사용량 계획·API 키, 캐싱
고르는 법	기본값. 대부분 여기서 충분	WAF·API 키·29초 초과 처리가 필요할 때

합정 — **페이로드 상한 10MB는 양쪽 다 상향 불가**다. 파일 업로드는 API Gateway를 통과시키지 말고 **S3 Presigned URL**로 클라이언트가 직접 올리게 하라. 이게 비용·지연·안정성 모두에서 정답이다.

03 / STORAGE

스토리지 — 파일을 어디에 둘 것인가

S3(오브젝트) · EBS(블록) · EFS(파일). 셋을 가르는 질문은 하나다 — 여러 서버가 동시에 같은 데이터를 *파일시스템처럼* 봐야 하는가.

S3 Simple Storage Service `리전 (이름은 글로벌 고유)`

오브젝트 스토리지. HTTP API로 넣고 꺼낸다. 파일시스템이 아니라서 부분 수정이 안 되고, 통째로 덮어쓴다. 2020년 이후 **쓰기 후 읽기 강한 일관성이 기본**이라 "방금 올린 파일이 안 보이는" 문제는 이제 없다.

스토리지 클래스
: Standard · Intelligent-Tiering(접근 패턴 모르면 이것) · Standard-IA · One Zone-IA · Express One Zone(초저지연) · Glacier Instant / Flexible / Deep Archive

언제
: 이미지·첨부파일, 정적 웹 호스팅, 로그·백업 아카이브, 데이터 레이크 원본, Presigned URL 직업로드.

수치
: 오브젝트 최대 50TB(2025-12 상향, 이전 5TB) · 단일 PUT은 5GB까지, 그 이상은 멀티파트 업로드

합정
: **요청 수 과금과 리스팅 비용을 과소평가한다.** 작은 파일 수백만 개를 Glacier로 넘기면 오브젝트당 전환 요청비가 저장비를 초과할 수 있다. 그리고 **수명주기 규칙에 미완료 멀티파트 업로드 정리(AbortIncompleteMultipartUpload)**를 반드시 넣어라 — 실패한 업로드 조각이 보이지 않는 곳에 쌓여 계속 과금된다.

EBS Elastic Block Store `AZ`

EC2에 붙이는 네트워크 블록 디바이스. 물리 디스크처럼 포맷하고 마운트한다. **AZ에 묶여 있어** 다른 AZ 인스턴스에는 붙일 수 없다(스냅샷을 거쳐야 한다).

타입
: `gp3` — 기본값. IOPS·처리량을 용량과 독립적으로 설정 가능해 gp2보다 싸고 유연하다. `io2 Block Express` — 고IOPS DB용. `st1/sc1` — HDD, 순차 대용량.

합정
: **인스턴스를 종료해도 스냅샷은 남는다.** 그리고 대부분 아무도 안 지운다. 계정 몇 년 쓰면 정체불명 스냅샷이 수백 개 쌓여 있다. 또 아직 `gp2`를 쓰고 있다면 **gp3로 바꾸는 것만으로 대체로 20% 절감**된다 — 다운타임 없이 볼륨 수정으로 전환 가능하다.

EFS Elastic File System `리전 (다중 AZ)`

NFS 기반 공유 파일시스템. 여러 EC2·ECS·Lambda가 *동시에* 같은 경로를 읽고 쓸 수 있다.

언제
: 레거시 앱이 로컬 파일 공유를 전제로 짜여 있을 때, WordPress 업로드 디렉터리, ML 학습 데이터 공유 마운트.

합정
: **EFS는 S3보다 GB당 몇 배 비싸다.** "그냥 파일 저장소"가 필요한 거라면 EFS가 아니라 S3다. EFS는 *POSIX 파일시스템 시맨틱이 반드시 필요할 때만* 쓰는 물건이다. 그리고 NFS 지연은 로컬 디스크와 비교할 수 없으므로 소파일 대량 I/O에는 부적합하다.

---

04 / DATABASE

데이터베이스 — 상태를 어디에 저장할 것인가

"확장성" 때문에 DynamoDB를 고르는 판단은 대체로 틀린다. 결정적 기준은 **액세스 패턴이 미리 고정되어 있는가**다. 나중에 임의의 조건으로 조회해야 한다면 관계형이 맞다.

DB 선택 요약

선택	맞는 상황	피해야 할 상황
RDS	표준 MySQL/PostgreSQL. 특별한 요구 없는 일반 서비스	초당 수만 쓰기, 밀리초 이하 지연 요구
Aurora	RDS 호환 + 읽기 복제 15개, 스토리지 자동 확장, 빠른 장애 복구	트래픽이 작아 비용 정당화가 안 될 때
DynamoDB	키 기반 조회가 확정적. 세션·카트·이벤트 로그·리더보드	애드혹 쿼리, JOIN, 집계 리포트
ElastiCache	캐시, 세션 저장, 레이트 리밋, 분산 락, 랭킹	영속 저장(원본 데이터를 여기만 두면 안 된다)

RDS Relational Database Service

리전 · AZ

MySQL·PostgreSQL·MariaDB·Oracle·SQL Server를 관리형으로. 백업·패치·장애 조치를 AWS가 맡는다.

핵심 옵션

Multi-AZ — 다른 AZ에 동기 스탠바이. 가용성용이지 성능용이 아니다. **읽기 복제본** — 비동기 복제, 읽기 부하 분산용. 둘은 목적이 다르며 보통 함께 쓴다.

합정

> **읽기 복제본은 복제 지연이 있다.** 쓰고 바로 복제본에서 읽으면 없는 데이터를 보게 된다. "저장했는데 목록에 안 나와요" 버그의 단골 원인이다. 그리고 **Multi-AZ 페일오버는 DNS 전환**이라 수십 초가 걸린다 — 애플리케이션에 커넥션 재시도 로직이 없으면 그 시간만큼 그대로 장애다.

Aurora AWS 자체 개발 클라우드 네이티브 엔진 리전

MySQL·PostgreSQL 호환이면서 스토리지 계층을 분산 재설계했다. 컴퓨트와 스토리지가 분리되어 복제본 추가가 빠르고, 스토리지는 자동으로 늘어난다.

SERVERLESS V2 — ACU 단위(0.5씩)로 자동 스케일. **0 ACU까지 내려가 자동 일시정지**가 가능해졌다 — 개발·스테이징 환경 비용이 크게 줄어든다.

합정 — **I/O 과금 모델을 확인하라.** 기본 구성은 요청한 I/O만큼 별도 과금되어, I/O가 많은 워크로드는 예상 청구서를 크게 벗어난다. I/O 비중이 높다면 `Aurora I/O-Optimized` 구성이 총액에서 더 쌀 수 있다. 사용 패턴을 보고 고르는 것이지 기본값이 항상 옳지 않다.

DynamoDB 관리형 NoSQL 키-값/문서 DB 리전

파티션 키로 샤딩하고 한 자릿수 밀리초 지연을 유지한다. **현재 온디맨드가 기본이자 권장 모드**이며, 트래픽이 예측 가능하고 안정적일 때만 프로비저닝이 더 싸다.

설계 원칙 — 테이블을 먼저 만들지 말고 *쿼리 목록을 먼저 적어라*. 그 쿼리들이 PK/SK와 GSI 설계를 결정한다. 관계형처럼 정규화하면 반드시 실패한다.

부가 기능 — DynamoDB Streams(변경 이벤트 → Lambda) · TTL(자동 만료 삭제) · 조건부 쓰기(낙관적 락)

합정 — **핫 파티션.** 파티션 키를 `date`나 `status` 같은 저카디널리티 값으로 잡으면 트래픽이 한 파티션에 몰려, 테이블 전체 용량이 남아도는데도 스로틀된다. 그리고 `Scan`은 테이블 전체를 읽는다 — 프로덕션 코드에 Scan이 있다면 그건 설계가 잘못됐다는 신호다.

ElastiCache 관리형 인메모리 캐시 리전 · AZ

엔진은 **Valkey · Redis OSS · Memcached** 세 가지. Valkey는 Redis의 오픈소스 포크로, AWS 기준 **노드 기반 20%, 서버리스 33% 저렴**하다. 신규 구축이면 Valkey가 합리적 기본값이다.

언제 — DB 조회 결과 캐싱, 세션 스토어, 레이트 리미터, 분산 락, Sorted Set 기반 실시간 랭킹.

합정 — **캐시 스탬피드.** 인기 키의 TTL이 만료되는 순간 수천 요청이 동시에 DB로 쏟아진다. TTL에 지터를 주거나 만료 직전 백그라운드 갱신(refresh-ahead)으로 막는다. 그리고 ElastiCache는 **휘발성**이다 — 원본 없이 여기에만 있는 데이터는 언젠가 사라진다고 가정하라.

05 / MESSAGING & EVENTS

메시징·이벤트 — 서비스를 어떻게 분리할 것인가

이 카테고리를 이해하는 순간 아키텍처가 달라진다. 동기 호출을 비동기로 바꾸면 장애가 전파되지 않고, 부하가 큐에 흡수된다.

네 서비스의 결정적 차이

서비스	모델	소비자	대표 용도
SQS	Pull — 소비자가 폴링	보통 1종류(경쟁 소비)	작업 큐, 부하 완충, 재시도
SNS	Push — 발행 즉시 전달	N개 구독자에 팬아웃	알림, 한 이벤트를 여러 곳에 동시 전파
EventBridge	규칙 기반 라우팅	내용으로 필터링해 타깃 선택	이벤트 버스, SaaS·AWS 이벤트 연동, 크론
Kinesis Data Streams	순서 있는 스트림, 재생 가능	독립 소비자 다수가 각자 위치 관리	클릭스트림, 실시간 분석, CDC

SQS Simple Queue Service 리전

메시지를 쌓아두고 소비자가 가져간다. 생산자와 소비자의 속도 차이를 흡수하는 완충재.

두 종류 **Standard** — 사실상 무제한 처리량, 순서 보장 없음, 최소 1회 전달(중복 가능). **FIFO** — 순서 보장 + 정확히 1회 처리. 기본 `300` TPS(배치 3,000), 고처리량 모드에서 리전별 최대 `70,000` TPS(us-east-1·us-west-2·eu-west-1 기준).

수치 메시지 최대 `1MiB`(과거 256KB에서 상향) · 보관 최대 `14`일(기본 4일) · 가시성 타임아웃 최대 12시간

합정

> **DLQ 없이 큐를 만들지 마라.** 처리 실패한 메시지가 무한 재시도되며 소비자를 계속 죽이는 독약 메시지(poison message) 문제는 DLQ가 없으면 잡을 방법이 없다. 그리고 **Standard 큐는 중복이 온다** — 소비자는 반드시 멱등해야 한다. FIFO 고처리량 70,000 TPS는 *메시지 그룹 ID를 충분히 분산했을 때*의 큐 전체 한도로, 단일 그룹의 파티션 한도는 여전히 300/3,000이다.

SNS Simple Notification Service 리전

토픽에 발행하면 모든 구독자에게 동시에 밀어 넣는다. 구독 대상은 SQS·Lambda·HTTP 엔드포인트·이메일·SMS·모바일 푸시.

대표 패턴 **Fan-out** — SNS 토픽 하나에 SQS 큐 여러 개를 구독시킨다. "주문 완료" 한 번 발행으로 결제·재고·알림 서비스가 각자의 큐에서 독립적으로 처리하고, 하나가 죽어도 나머지는 멀쩡하다.

함정 > **SNS는 저장하지 않는다.** 구독자가 그 순간 못 받으면(재시도 정책 소진 후) 메시지는 사라진다. 유실이 곤란하면 SNS → SQS로 받아 큐에 남겨라. Lambda를 SNS에 직접 붙이는 구성은 편하지만 유실 위험을 안는다.

EventBridge 서버리스 이벤트 버스 리전

이벤트의 *내용*을 보고 규칙에 맞는 타깃으로 보낸다. SNS가 "누가 구독했나"라면 EventBridge는 "이 이벤트가 조건에 맞나"다.

언제 AWS 서비스 자체 이벤트 구독(EC2 상태 변화, CodePipeline 실패 등) · Scheduler로 크론 대체 · SaaS(Datadog, Shopify 등) 이벤트 수신 · 계정 간 이벤트 공유 · 마이크로서비스 도메인 이벤트 버스.

함정 > **규칙이 안 맞으면 이벤트는 조용히 버려진다.** 에러도 로그도 없다. 패턴 매칭 오타 하나로 파이프라인 전체가 멈춰도 아무 알림이 없으므로, 규칙마다 `FailedInvocations` 지표에 알람을 걸고 매칭 안 되는 이벤트를 잡을 catch-all 규칙을 하나 두는 게 안전하다.

Kinesis Data Streams 실시간 스트리밍 리전

샤드 단위로 순서를 보장하며, 데이터를 보관 기간 동안 남겨둬 **여러 소비자가 각자 재생**할 수 있다. 이 점이 SQS와의 결정적 차이다.

이웃 서비스 **Amazon Data Firehose**(구 Kinesis Data Firehose, 2024년 개명) — 소비자 코드 없이 S3·Redshift·OpenSearch로 버퍼링해 적재. **Amazon MSK** — 관리형 Apache Kafka. Kafka API·생태계가 필요할 때.

함정 > **Firehose는 실시간이 아니다.** 버퍼 크기/시간 단위로 모아서 보내므로 초 단위 지연이 있다. 대시보드에 "왜 데이터가 1분 늦지?"의 답은 대개 버퍼 설정이다. 그리고 단순 작업 큐가 필요한 거라면 Kinesis가 아니라 SQS다 — 샤드 관리 비용을 이유 없이 지불하지 마라.

Step Functions 워크플로 오케스트레이션 리전

여러 Lambda·서비스 호출을 상태 머신으로 엮는다. 재시도·분기·병렬·에러 처리를 코드가 아니라 정의로 표현하고, 각 단계 실행 이력이 콘솔에 남는다.

두 유형 — **Standard** — 최대 1년 실행, 실행당 과금, 정확히 1회. **Express** — 최대 5분, 대량 고빈도용, 훨씬 저렴.

언제 — 주문 처리 사가(보상 트랜잭션 포함), ETL 파이프라인, 사람 승인이 끼는 장기 프로세스, Lambda 15분 한계를 넘는 작업의 분할.

합정 — **상태 전이마다 과금된다.** 루프를 상태 머신으로 수만 번 돌리면 Lambda 비용보다 Step Functions 비용이 커진다. 반복이 많은 로직은 Standard가 아니라 Express를 쓰거나, 애초에 코드 안에서 처리하는 게 맞다.

---

06 / SECURITY & IDENTITY

보안·자격증명 — 누가 무엇을 할 수 있는가

여기서의 실수는 조용하다. 잘못된 권한은 에러를 내지 않고, 문제가 드러날 땐 이미 늦다.

IAM Identity and Access Management 글로벌

모든 AWS API 호출은 IAM 평가를 거친다. 기본 규칙은 **명시적 Deny > 명시적 Allow > 암묵적 Deny**.

구성 — **User**(사람·장기 자격증명) · **Role**(임시 자격증명, 위임) · **Policy**(JSON 권한 문서) · **Group**(사용자 묶음)

현재 권장 — AWS 공식 모범 사례는 **사람은 IAM Identity Center로 페더레이션**, **워크로드는 롤(임시 자격증명)**, **장기 액세스 키는 회피**다. EC2엔 인스턴스 프로파일, EKS엔 IRSA/Pod Identity, Lambda엔 실행 롤 — 어디에도 액세스 키를 심지 않는다.

합정 — `*`로 시작한 권한은 절대 좁혀지지 않는다. "일단 열고 나중에 조이자"는 조여진 적이 없다. IAM Access Analyzer로 *실제 사용된 권한* 기반 정책을 생성하는 습관을 들여라. 그리고 IAM은 글로벌 서비스지만 **컨트롤 플레인이 us-east-1에 있다** — us-east-1 장애 시 다른 리전에서도 IAM 변경이 막힐 수 있다.

KMS Key Management Service 리전

암호화 키의 생성·보관·회전·감사를 담당한다. S3·EBS·RDS·Secrets Manager의 암호화가 전부 여기에 기댄다.

비용 고객 관리형 키 $1/월/키 · 대칭 요청 $0.03/1만 건 · AWS 관리형 키는 생성·보관 무료(요청은 과금)

합정 **KMS 키 정책과 IAM 정책은 둘 다 통과해야 한다.** IAM에서 `kms:Decrypt`를 줬는데도 거부되는 이유는 대개 키 정책에 해당 주체가 없어서다. 그리고 **키를 삭제하면 그 키로 암호화한 데이터는 영구히 복구 불가**다. 삭제 대기 기간(7~30일)이 있는 이유가 그것이다.

Secrets Manager / SSM Parameter Store 비밀·설정 저장소 리전

둘 다 값을 안전하게 저장하지만 비용과 기능이 다르다. 무턱대고 Secrets Manager를 쓰면 돈이 샌다.

	Secrets Manager	Parameter Store
비용	$0.40/비밀/월 + $0.05/1만 API	**Standard 무료** / Advanced $0.05/월
자동 회전	지원 (RDS 등 내장 통합)	없음
쓰는 때	DB 비밀번호, 회전이 필요한 API 키	환경 설정값, 회전 불필요한 토큰, 피처 플래그

합정 Parameter Store의 `SecureString`도 KMS로 암호화된다. **회전이 필요 없는 값에 Secrets Manager를 쓰는 건 낭비**다. 비밀 100개면 연 $480인데 Parameter Store Standard였다면 0원이다. 반대로, 매 요청마다 비밀을 조회하는 코드는 API 호출비가 붙으니 캐싱하라.

Cognito 사용자 인증·인가 리전

둘로 나뉜다. **User Pool**은 사용자 디렉터리이자 OIDC IdP로 ID/액세스 토큰을 발급한다. **Identity Pool**은 그 토큰을 STS 임시 AWS 자격증명으로 교환한다(게스트 접근 포함).

언제 — 소셜 로그인(Google·Apple)과 자체 회원가입을 함께 붙일 때, 모바일 앱이 S3에 직접 업로드해야 할 때(Identity Pool 경유), ALB 앞단 인증.

함정 — **과금 체계가 2024-11-22에 바뀌었다.** Lite / Essentials / Plus 3단계로 나뉘고 **무료 MAU가 50,000에서 10,000으로 축소**됐다(이전 생성 풀은 기존 조건 유지, Plus는 무료 구간 없음). 예전 블로그의 "5만 MAU 무료"를 믿고 설계하면 안 된다. Identity Pool 자체는 무료다.

WAF / Shield / ACM 웹 방어와 인증서 글로벌 · 리전

Shield Standard는 모든 계정에 자동 적용되는 무료 L3/L4 DDoS 방어다. 별도로 켤 것이 없다. **WAF**는 L7 규칙(SQLi·XSS·레이트 리밋·지역 차단)을 ALB·CloudFront·API Gateway에 붙인다. **ACM**은 AWS 서비스에 붙이는 공인 인증서를 무료로 발급하고 자동 갱신한다.

비용 — WAF Web ACL $5/월 + 규칙당 $1/월 + 요청 100만당 $0.60 · Shield Advanced는 $3,000/월(1년 약정)

함정 — **CloudFront에 붙일 ACM 인증서는 반드시 us-east-1에서 발급해야 한다.** 서울 리전에서 만든 인증서는 CloudFront 배포에서 선택지에 아예 나타나지 않는다. 실무에서 가장 많이 시간을 잡아먹는 함정 중 하나다. 참고로 **ACM에서 내보내기(export) 가능한 인증서는 유료**(FQDN $7, 와일드카드 $79)이니 무료라는 표현은 "AWS 서비스에 붙여 쓰는 경우"에 한정된다.

07 / DEPLOY & OBSERVE

배포·관측 — 어떻게 올리고 어떻게 볼 것인가

콘솔에서 손으로 만든 인프라는 재현이 안 되고, 로그가 없는 서비스는 장애 원인을 영원히 못 찾는다.

`CloudFormation / CDK` Infrastructure as Code 리전

CloudFormation은 YAML/JSON 템플릿으로 리소스를 선언한다. **CDK**는 TypeScript·Python·Java·C#·Go·JavaScript로 코드를 써서 CloudFormation 템플릿을 생성한다 — 현재 **v2**가 유일한 지원 버전(v1은 2023-06 지원 종료)이다.

고르는 법 — 루프·조건·재사용 컴포넌트가 필요하면 CDK. 단순하고 변화 적은 스택이면 CloudFormation 원본. 멀티클라우드·기존 자산이 있으면 Terraform.

함정 — **드리프트.** IaC로 만든 리소스를 콘솔에서 급하게 고치는 순간 상태가 어긋나고, 다음 배포에서 그 수정이 조용히 되돌아간다. 그리고 CDK는 추상화가 높아 **실제로 생성되는 리소스를 모른 채 배포**하기 쉽다 — `cdk diff`를 반드시 읽고 머지하라.

ECR Elastic Container Registry 리전

컨테이너 이미지 저장소. 프라이빗과 퍼블릭(ECR Public Gallery) 양쪽을 제공한다. 프라이빗 저장 비용은 $0.10/GB/월.

함정 — **수명주기 정책을 안 걸면 이미지가 무한히 쌓인다.** CI가 커밋마다 이미지를 푸시하면 수백 GB가 되는 건 순식간이다. "태그 없는 이미지는 7일 후 삭제, 최근 30개만 보관" 같은 규칙을 *처음 리포지토리를 만들 때* 함께 넣어라.

`CloudWatch` 지표 · 로그 · 알람 리전

Metrics(수치 시계열) · Logs(로그 수집·검색) · Alarms(임계값 알림) · Logs Insights(쿼리)로 구성된다. Lambda·ECS 로그는 자동으로 여기 들어온다.

비용 — 수집 Standard $0.50/GB(첫 10TB, 이후 계단식 인하) · Infrequent Access 클래스는 그 절반 · 저장 $0.03/GB/월

함정 — **로그 그룹 보존 기간의 기본값은 "만료 없음"이다.** 아무도 안 보는 디버그 로그가 몇 년째 쌓이며 계속 과금된다. 로그 그룹을 만들 때 보존 기간을 반드시 지정하라. 그리고 `console.log`로 요청 본문을 통째로 찍는 습관은 수집 비용과 개인정보 유출을 동시에 만든다. **로그 클래스는 로그 그룹 생성 후 변경할 수 없다**는 점도 기억해 둘 것.

X-Ray / Application Signals 분산 추적과 APM 리전

요청이 여러 서비스를 지나며 어디서 얼마나 걸렸는지 추적한다. 마이크로서비스에서 "느린데 어디가 느린지 모르겠다"의 유일한 답.

중요한 전환

AWS는 계측 표준을 OpenTelemetry로 옮기고 있다. **X-Ray SDK와 X-Ray 데몬은 2026-02-25부터 유지보수 모드**(보안 수정만, 신규 기능 없음)에 들어갔다. 신규 프로젝트는 **OpenTelemetry SDK / ADOT**로 계측하고 **CloudWatch Application Signals**로 보는 구성이 권장 경로다.

합정

> **지금 X-Ray SDK로 새로 짜면 곧 마이그레이션 부채가 된다.** 기존 코드는 당장 문제없지만, 신규 서비스에 X-Ray SDK를 도입하는 판단은 2026년 기준으로는 재검토 대상이다. 샘플링 비율도 확인하라 — 기본 샘플링에서는 문제 요청이 추적에 안 잡힐 수 있다.

CloudTrail API 감사 로그 리전 · 글로벌 이벤트

"누가, 언제, 어떤 API를, 어디서 호출했는가"를 기록한다. CloudWatch가 *애플리케이션이 무엇을 했는가*라면 CloudTrail은 *사람과 롤이 AWS에 무엇을 했는가*다.

언제

"이 보안 그룹 누가 열었어?" "이 인스턴스 누가 지웠어?"에 답할 때. 컴플라이언스 감사. 침해 사고 조사.

합정

> **콘솔의 이벤트 기록은 90일치만 보인다.** 그 이상 보관하려면 S3로 내보내는 트레일을 직접 만들어야 한다. 사고가 터진 뒤에 만들면 이미 늦다. 그리고 S3 객체 읽기 같은 **데이터 이벤트는 기본적으로 기록되지 않으며** 별도로 켜야 하고, 양이 많아 비용이 크게 늘 수 있다.

08 / COST

비용 함정 정리

신입 개발자가 AWS 청구서를 터뜨리는 경로는 매번 같다. 아래 항목은 전부 위 본문에서 나온 것을 한 곳에 모은 것이다.

먼저 확인할 것

1. **퍼블릭 IPv4 주소는 2024-02-01부터 유료다.** 사용 중이든 유휴든 시간당 $0.005, 주소당 월 약 $3.6. 안 쓰는 Elastic IP를 방치하는 게 이제는 실제 비용이다.
2. **NAT Gateway.** 시간당 $0.045 + 처리 GB당 $0.045. AZ마다 두면 고정비만 월 $100+. S3·DynamoDB용 Gateway 엔드포인트(무료)를 먼저 붙여라.
3. **CloudWatch 로그 보존 무제한.** 기본값이 "만료 없음"이다. 로그 그룹마다 보존 기간을 지정하라.
4. **S3 미완료 멀티파트 업로드.** 콘솔 목록에 안 보이면서 과금된다. 수명주기 규칙으로 정리.
5. **고아 EBS 스냅샷과 미연결 볼륨.** 인스턴스를 지워도 남는다. `gp2`는 `gp3`로 바꾸면 대체로 20% 절감.
6. **회전 불필요한 값에 Secrets Manager.** 비밀당 $0.40/월. SSM Parameter Store Standard는 무료다.
7. **ALB/NLB, Aurora, ElastiCache 개발 환경 상시 기동.** 트래픽 0이어도 시간당 과금. Aurora Serverless v2는 이제 0 ACU까지 내려가 자동 일시정지가 되므로 개발 환경에 적합하다.

반대 관점 — 이 문서를 그대로 믿지 말아야 할 지점

1. 여기 적힌 **가격과 한도는 us-east-1 기준이고 리전마다 다르다.** 서울(ap-northeast-2)은 대체로 더 비싸다. 실제 설계 전에 해당 리전 요금 페이지를 직접 확인하라.
2. "서버리스가 기본값"은 트래픽 패턴에 달렸다. **상시 고부하라면 EC2 + Savings Plan이 Lambda·Fargate보다 몇 배 싸다.** 서버리스의 이점은 비용이 아니라 운영 부담 제거다.
3. SQS·EventBridge로 비동기 분리를 하면 결합도는 낮아지지만 **디버깅 난이도와 최종 일관성 문제가 그만큼 올라간다.** 서비스가 3개인 팀에 이벤트 버스를 도입하는 건 대개 과설계다.

본문 수치는 2026-07-29에 AWS 공식 문서·요금 페이지에서 확인했다. 요금과 서비스 한도는 자주 바뀌므로 설계 결정 전 원문 재확인을 권한다.

주요 출처

- Lambda quotas · EC2 instance type names · API Gateway REST quotas

- VPC pricing (NAT · public IPv4) · ALB · Route 53 routing policies · Route 53 pricing
- S3 storage classes · S3 50TB object size · S3 strong consistency
- SQS high throughput FIFO · SQS message quotas · SNS vs SQS vs EventBridge
- DynamoDB capacity modes · Aurora Serverless v2 · ElastiCache for Valkey
- IAM best practices · Cognito pricing · Secrets Manager pricing · ACM pricing
- X-Ray → OpenTelemetry 전환 · AWS CDK v2 · CloudWatch pricing
- CloudFront 무효화 과금(월 1,000 경로 무료) · ALB 유휴 타임아웃 기본 60초 · EBS gp3 (gp2 대비 최대 20% 저렴)